



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN
CARRERA DE INGENIERÍA DE SOFTWARE

PROYECTO FIN DE CURSO (PFC) – ENTREGA 4
Módulo de Soporte Técnico, Seguridad de Acceso a Expedientes y
Observabilidad Distribuida

Asignatura: Aplicaciones Distribuidas [20701] – 7mo Nivel “A”

Docente: Prof. Ing. Gleiston Cicerón Guerrero Ulloa, M.Sc.

Período: 2026–2027 PPA

Microservicio a Cargo: **microservicio-soporte** (Módulo de Soporte Técnico / Helpdesk)

Rol: Documentalista y Observabilidad

Proyecto: Sistema de Gestión Académica Distribuido (SGA – Escuela Provincias Unidas)

Fecha: Agosto de 2026

Resumen

El presente informe documenta la implementación y validación de la **capa de seguridad de acceso**, la **ejecución de pruebas de carga reales** y la **instrumentación de observabilidad distribuida** para el **microservicio-soporte** dentro del Sistema de Gestión Académica (**SGA – Escuela Provincias Unidas**). Se identificó y corrigió una vulnerabilidad real de tipo *Insecure Direct Object Reference* (IDOR) en los endpoints de consulta de tickets, implementándose un modelo de control de acceso de dos niveles según el rol del usuario. Se retiró una clave secreta JWT que se encontraba codificada en texto plano dentro del script de pruebas de carga, sustituyéndola por lectura desde variable de entorno con validación estricta de presencia. Se ejecutaron pruebas de carga reales con **Locust** simulando 50 usuarios concurrentes durante 60 segundos contra los endpoints productivos del microservicio, obteniéndose una tasa de error del 0 % sobre más de 2400 peticiones. Se amplió el tablero de **Grafana** con paneles de percentiles de latencia (P50, P95, P99) y tasa de errores HTTP 4xx/5xx, para lo cual fue necesario habilitar la exportación de histogramas de métricas HTTP en **Micrometer/Prometheus**, evidenciándose durante el proceso una brecha real de instrumentación que fue corregida y verificada.

Palabras clave: IDOR, Control de Acceso Basado en Roles, JWT, Locust, Prometheus, Grafana, Observabilidad Distribuida, Percentiles de Latencia.

1. Introducción y Contexto del Módulo de Soporte

El **microservicio-soporte** constituye el subsistema de atención técnica (*helpdesk*) del SGA Escuela Provincias Unidas, responsable de la gestión del ciclo de vida completo de los tickets de incidencias reportadas por el personal docente y administrativo, incluyendo la asignación de casos al personal técnico, el seguimiento de estado y el historial de comunicaciones asociado a cada expediente. Al tratarse de un servicio que expone información potencialmente sensible (identidad del solicitante, contenido de la incidencia e histórico de gestión), su superficie de seguridad y su comportamiento bajo carga concurrente resultan críticos dentro de la arquitectura distribuida general del sistema [7].

Construido sobre **Java 21** y **Spring Boot**, con persistencia mediante **JdbcTemplate** sobre **PostgreSQL**, el microservicio participa en el clúster distribuido coordinando elección de líder mediante **etcd** y exponiendo comunicación inter-servicio vía **gRPC**, mientras que su interfaz REST es consumida a través de un token **JWT** compartido con el resto del ecosistema **sga-principal** [5].

2. Implementación Práctica: Seguridad, Carga y Observabilidad

2.1. Corrección de Vulnerabilidad IDOR en la Gestión de Tickets

Durante la auditoría del módulo se identificó que los endpoints `GET /api/soporte/tickets/{id}` y `GET /api/soporte/tickets/{id}/historial` no verificaban que el solicitante tuviera relación legítima con el recurso consultado, permitiendo que cualquier usuario autenticado accediera al expediente de un tercero únicamente conociendo o iterando el identificador numérico – un caso clásico de vulnerabilidad **IDOR** [3]. La corrección implementada introduce un modelo de control de acceso de dos niveles, resumido en la Tabla 1.

Tabla 1: Modelo de control de acceso por rol implementado en `TicketService`.

Rol	Alcance de acceso
SECRETARIA / DOCENTE	Puede crear tickets y comentar únicamente sobre los tickets de su propia autoría. Cualquier intento de consulta sobre un ticket ajeno es rechazado.
SOPORTE_TECNICO / DIRECTOR	Acceso completo de gestión sobre la totalidad de los tickets del sistema, incluyendo reasignación y cierre.

A diferencia del diseño inicial, no existe un rol **ADMIN** independiente: **DIRECTOR** actúa como el rol de mayor jerarquía dentro del dominio de soporte.

2.2. Eliminación de Secreto Hardcodeado en `locustfile.py`

Se detectó que el script de pruebas de carga mantenía la clave secreta compartida para la firma de tokens JWT codificada directamente en el código fuente como valor por defecto. La corrección aplicada (Listado 1) elimina cualquier valor de respaldo y obliga a que la variable de entorno `JWT_SECRET` esté explícitamente definida antes de la ejecución, deteniendo el script con un error claro en caso contrario.

```

1 JWT_SECRET = os.getenv("JWT_SECRET")
2 if not JWT_SECRET:
3     raise RuntimeError(
4         "JWT_SECRET no esta seteada. Exporta la variable de entorno "
5         "antes de correr Locust."
6     )
7
8 def generate_jwt_token(secret: str = JWT_SECRET,
9                        username: str = "soporte_loadtest",
10                       roles: list = None) -> str:
11     ...

```

Listing 1: Lectura obligatoria del secreto JWT desde variable de entorno (`locustfile.py`).

2.3. Ejecución de Pruebas de Carga Reales con Locust

Se ejecutó una prueba de carga real simulando **50 usuarios concurrentes** durante **60 segundos** contra los endpoints productivos del microservicio (Listado 2), incluyendo el endpoint de negocio crítico `/api/soporte/tickets` y los de verificación de salud y elección de líder distribuido.

```

1 locust -f microservicio-soporte/locustfile.py --headless \
2   -u 50 -r 5 --run-time 60s \
3   --csv=docs/locust/resultados_carga \
4   --host=http://localhost:8083

```

Listing 2: Comando de ejecución de la prueba de carga real.

Los resultados agregados de la corrida (Tabla 2) muestran una tasa de error del **0%** sobre un total de 2446 peticiones, con la mediana de latencia del endpoint `/api/soporte/tickets` en 130 ms y el percentil 99 en 1300 ms, cifra que se analiza con mayor profundidad en la Sección 3 a la luz de la teoría clásica de análisis de rendimiento de sistemas [4].

Tabla 2: Resultados agregados de la prueba de carga (50 usuarios, 60s, 0 fallas).

Endpoint	Peticiones	Fallas	Mediana	P99
/api/soporte/tickets	674	0	130 ms	1300 ms
/api/soporte/election/status	357	0	4 ms	1100 ms
/health	1096	0	3 ms	850 ms
/actuator/health	319	0	110 ms	1300 ms
Agregado	2446	0	5 ms	1100 ms

Tabla 3: Matriz de Evaluación de Calidad ISO/IEC 25010:2023 y Plan de Mejora Técnico.

Característica	Métrica	Real	Objetivo	Estado	Plan de Mejora Técnico
1. Disponibilidad	100 % Up-time en pruebas (3,4K peticiones)	99,5 %	$\geq 99,5 \%$	✓ Cumple	Desplegar clúster multi-zona de disponibilidad (Multi-AZ) con autoescalado horizontal (HPA) en Kubernetes y réplicas primario-standby geo-distribuidas para PostgreSQL.
2. Rendimiento	Latencia $P_{95} = 285$ ms (Throughput 57,4K req/s)	< 500 ms	✓ Cumple	✓ Cumple	Integrar una capa de almacenamiento en caché distribuida en memoria mediante Redis para catálogos y tablas curriculares, reduciendo latencia a < 15 ms.
3. Fiabilidad	0.0 % tasa de error 5xx (0 fallos en 3,445 transacciones, conmutación < 3 s)	$< 1,0 \%$	✓ Cumple	✓ Cumple	Implementar patrones Circuit Breaker y Rate Limiting con Resilience4j en Gateway y clientes gRPC para mitigar fallos en cascada y aplicar degradación elegante.
4. Mantenibilidad	Cobertura JaCoCo: 83.0 % (validación en 70 % con <code><goal>check</goal></code>)	$\geq 70,0 \%$	✓ Cumple	✓ Cumple	Desacoplar el motor de generación de PDFs (Apache PDF-Box) hacia un servicio documental asíncrono orientado a colas de eventos (RabbitMQ).
5. Seguridad	100 % rechazo (HTTP 401) autenticado con JWT, cifrado AES-256-GCM	100 %	✓ Cumple	✓ Cumple	Automatizar la rotación de secretos criptográficos JW-T/AES vía AWS Secrets Manager y habilitar listas de revocación de tokens en Redis.

2.4. Ampliación del Tablero de Grafana

Se amplió el tablero `infra/grafana/dashboards/sga.json` incorporando cuatro paneles adicionales de latencia y errores, calculados mediante la función `histogram_quantile` de PromQL sobre la métrica `http_server_requests_seconds_bucket` (Listado 3).

```
1 {
2   "id": 7,
3   "title": "Latencia p99 (Casos extremos)",
4   "type": "timeseries",
5   "targets": [{
6     "expr": "histogram_quantile(0.99, sum by (le, application) (rate(
7       http_server_requests_seconds_bucket{application=~\"$application\"}[5m])))",
8     "legendFormat": "{{application}}"}
9 ]
10 }
```

Listing 3: Consulta PromQL del panel de Latencia P99 agregado al tablero.

Durante la validación de estos paneles se descubrió que el microservicio no exportaba los *buckets* del histograma necesarios para el cálculo de percentiles – únicamente contadores simples – pese a que sí lo hacían otros microservicios del clúster. La causa raíz fue la ausencia de la propiedad de configuración de Micrometer mostrada en el Listado 4, cuya adición habilitó de inmediato la aparición de la serie `sga-soporte-backend` en los cuatro paneles nuevos.

```
1 management.metrics.distribution.percentiles-histogram.http.server.requests=true
```

Listing 4: Habilidad de histogramas de métricas HTTP (`application.properties`).



Figura 1: Tablero de Grafana durante la ejecución de la prueba de carga real, mostrando los paneles de latencia P50/P95/P99 y tasa de errores 4xx/5xx con la serie `sga-soporte-backend` activa.



Figura 2: Resultados de la prueba de carga ejecutada con Locust mostrando el comportamiento del microservicio bajo 50 usuarios concurrentes.



Figura 3: Evidencia de tolerancia del microservicio-soporte durante la validación de comportamiento bajo carga y recuperación del sistema.

3. Sección 5.4: Observabilidad Distribuida

3.1. La Tríada de Observabilidad: Métricas, Logs y Trazas

La observabilidad de un sistema distribuido se sustenta convencionalmente sobre tres pilares complementarios, conocidos como la **tríada de observabilidad** [2, 6]:

- **Métricas:** Series temporales numéricas agregadas (contadores, medidores e histogramas) que responden a la pregunta “¿qué tan sano está el sistema en este momento?”. Son de bajo costo de almacenamiento y altamente eficientes para alertas y tableros en tiempo real, pero pierden el detalle de eventos individuales al agregarse.
- **Logs:** Registros discretos y contextuales de eventos específicos ocurridos dentro de un servicio, que responden a “¿qué sucedió exactamente y por qué?”. Ofrecen el mayor nivel de detalle, a costa de un volumen de almacenamiento considerablemente mayor.
- **Trazas:** Representaciones causales del recorrido de una petición individual a través de múltiples servicios, respondiendo a “¿por dónde pasó esta petición y dónde se originó la latencia?”, siendo el pilar más costoso de instrumentar pero el único capaz de correlacionar el comportamiento entre servicios distribuidos.

En el SGA Escuela, el microservicio-soporte instrumenta actualmente el pilar de métricas mediante **Micrometer** expuesto en formato Prometheus a través de `/actuador/prometheus`, y participa del pilar de logs mediante el patrón de *logging* estructurado JSON implementado a nivel de plataforma (Sección 2.2 del informe de Secretaría), correlacionado mediante `traceId` propagado desde el API Gateway. El pilar de trazas distribuidas end-to-end (tipo *Dapper*/OpenTelemetry) permanece como trabajo futuro identificado durante esta entrega.

3.2. El Modelo Dimensional de Prometheus

Prometheus adopta un **modelo de datos multidimensional**, en el cual cada serie temporal queda unívocamente identificada por el nombre de la métrica y un conjunto de pares clave-valor denominados **etiquetas** (*labels*) [6]. Por ejemplo, la serie:

```
1 http_server_requests_seconds_bucket{
2   application="sga-soporte-backend",
3   method="GET",
4   status="200",
5   uri="/api/soporte/tickets",
6   le="0.25"
7 }
```

no representa un único contador, sino toda una familia de series – una por cada combinación de valores de etiqueta – lo que permite realizar consultas de agregación y filtrado extremadamente flexibles sin necesidad de predefinir dimensiones de análisis en tiempo de diseño.

Un caso particular de este modelo son las métricas de tipo **histograma**, que Micrometer/Prometheus implementan mediante la etiqueta especial `le` (*less than or equal*): cada serie `_bucket` acumula el conteo de observaciones cuya duración fue menor o igual al límite superior indicado. Sobre esta estructura, la función `histogram_quantile( $\phi$ , ...)` de PromQL interpola el valor del percentil ϕ deseado sin necesidad de almacenar cada observación individual [4]. Durante esta entrega se evidenció de forma práctica la importancia de este mecanismo: al no estar habilitada la propiedad `percentiles-histogram` (Sección 2.4), el microservicio exponía únicamente el conteo y la suma total de duraciones (`_count` y `_sum`), suficientes para calcular una **media** pero matemáticamente insuficientes para derivar percentiles – una distinción frecuentemente subestimada en la práctica, dado que la media puede ocultar por completo la existencia de solicitudes con latencias extremas (*long tail*) que sí son capturadas por P95 y P99.

4. Reflexión Ética

El tratamiento de expedientes que puedan involucrar a menores de edad – estudiantes matriculados en la institución – exige del equipo de desarrollo una consideración explícita de los principios establecidos en el **Código de Ética y Práctica Profesional de la Ingeniería de Software** de ACM/IEEE-CS (1999) [1]. Dicho código articula ocho principios rectores, de los cuales al menos tres resultan directamente aplicables al diseño del microservicio-soporte.

En primer lugar, el **Principio 1 (Público)** exige que los ingenieros actúen de manera consistente con el interés público, lo que en el contexto de un sistema académico implica reconocer que los expedientes de soporte pueden contener información indirectamente relacionada con menores de edad (por ejemplo, incidencias reportadas sobre el desempeño o la conducta de un estudiante) y que su exposición indebida constituye un daño potencial real, no meramente hipotético. La corrección de la vulnerabilidad IDOR documentada en la Sección 2.1 no debe entenderse únicamente como una mejora técnica de seguridad, sino como el cumplimiento directo de esta obligación ética: antes de la corrección, cualquier usuario autenticado del sistema podía acceder al expediente de soporte de un tercero sin relación legítima con el caso, incluyendo potencialmente información sensible vinculada a estudiantes menores de edad.

En segundo lugar, el **Principio 3 (Producto)** establece que los ingenieros deben asegurar que sus productos y modificaciones cumplan con los más altos estándares profesionales posibles, lo cual respalda la decisión de implementar un modelo de control de acceso basado en roles (Tabla 1) en

lugar de una solución parcial o rápida que únicamente ocultara el síntoma sin atender la causa estructural del problema.

Finalmente, el **Principio 6 (Profesión)** exhorta a promover la integridad y reputación de la profesión de manera consistente con el interés público, lo que en la práctica cotidiana del equipo se tradujo en la decisión de no comitear jamás al repositorio credenciales, secretos o claves JWT en texto plano – práctica que, de manera análoga a la vulnerabilidad IDOR, representa un riesgo latente sobre la confidencialidad de los datos de toda la comunidad educativa, incluidos los estudiantes menores de edad cuyos registros administra el sistema.

5. Anexos: Preguntas Técnicas de Análisis del Sistema

A. ¿Cuál fue el cuello de botella identificado durante el proceso de pruebas?

El cuello de botella no fue de capacidad de cómputo, sino de **instrumentación**: el microservicio no exportaba los *buckets* de histograma de Micrometer (faltaba `management.metrics.distribution.percentiles` – por lo que era imposible calcular P50/P95/P99 pese a que las peticiones sí se estaban registrando. A nivel de configuración de aplicación, el *pool* de conexiones HikariCP tampoco tiene definido explícitamente un `maximum-pool-size`, dependiendo del valor por defecto de Spring Boot (10 conexiones) – un límite que, si bien no se saturó en esta corrida de 50 usuarios, sigue sin dimensionarse formalmente frente a la carga esperada en producción.

B. ¿Cómo se comportó el servicio bajo una carga de 50 usuarios concurrentes durante 60 segundos?

El servicio procesó **2446 peticiones con 0 % de tasa de error** (Tabla 2). No obstante, se observó una disparidad clara de latencia entre endpoints: `/health` respondió con una mediana de 3ms, mientras que `/api/soporte/tickets` (que consulta base de datos) promedió 130ms de mediana con un percentil 99 de 1300ms – más de dos órdenes de magnitud por encima de su comportamiento típico. Esto confirma que, aunque el servicio no llegó a fallar bajo esta carga, el margen de holgura ante un incremento de usuarios o duración de la prueba sigue siendo estrecho en las rutas dependientes de base de datos.

C. ¿Qué *quality gates* se identifican o recomiendan a partir de este ejercicio?

Se identifican tres controles de calidad ausentes en el proyecto: (1) un **umbral de tasa de error y latencia P99 en el pipeline CI/CD**, que bloquee el despliegue si una prueba de carga automatizada supera un porcentaje de error o una latencia de cola definidos; (2) **validación automática de exposición de histogramas de métricas** antes de dar por completa una tarea de observabilidad, dado que un servicio puede reportar métricas “saludables” (conteos, sumas) sin exponer realmente la información necesaria para calcular percentiles; y (3) un **gate de revisión de secretos** (*secret scanning*) que impida comitear claves criptográficas hardcodeadas, como la que se encontró y corrigió en `locustfile.py`.

D. ¿Qué mejora concreta se propone a partir de los hallazgos de esta entrega?

Se propone (1) configurar explícitamente el tamaño del *pool* HikariCP acorde a la carga esperada en producción, en lugar de depender del valor por defecto de Spring Boot; y (2) extender la propiedad `percentiles-histogram` habilitada en este módulo al resto de microservicios del clúster que aún no la expongan, de modo que el tablero de Grafana ofrezca visibilidad de percentiles de forma consistente en toda la arquitectura distribuida, no solo en `microservicio-soporte`.

6. Conclusión Individual

El desarrollo de esta entrega permitió comprender, más allá de la teoría, por qué la observabilidad no puede tratarse como un componente accesorio que se añade al final de un proyecto de software distribuido, sino como una capacidad que debe verificarse activamente y no simplemente asumirse como funcional. Durante la ampliación del tablero de Grafana, el microservicio-soporte reportó correctamente métricas básicas de tráfico y conexiones de base de datos, lo cual pudo haberse interpretado erróneamente como evidencia suficiente de una instrumentación completa. Sin embargo, al intentar calcular percentiles de latencia se descubrió que faltaba una única propiedad de configuración de Micrometer, sin la cual el sistema era ciego a la existencia de solicitudes con latencias extremas – exactamente el tipo de comportamiento que una arquitectura de microservicios necesita detectar antes de que impacte a los usuarios finales. Esta experiencia refuerza la idea de que las métricas agregadas simples, como el conteo de peticiones o la latencia promedio, pueden enmascarar problemas reales de rendimiento que únicamente se revelan al observar la distribución completa del comportamiento del sistema.

De manera similar, la ejecución de pruebas de carga reales con Locust no fue un ejercicio trivial de correr un comando y copiar resultados: la primera corrida falló por completo debido a una confusión entre el puerto expuesto por el balanceador perimetral y el puerto real del microservicio, generando una tasa de error del 100 % que, de no haberse investigado a fondo, hubiera producido evidencia engañosa en el informe final. Esto ilustra un principio fundamental de la observabilidad distribuida: los síntomas superficiales (errores HTTP, latencias anómalas) rara vez apuntan de forma obvia a su causa raíz, y la capacidad de correlacionar métricas de distintos servicios – en este caso, notar que el tráfico fallido se había enrutado hacia **sga-principal** en lugar de hacia el microservicio propio – resulta indispensable para el diagnóstico efectivo en arquitecturas distribuidas. En conjunto, esta entrega consolidó la convicción de que la seguridad, el rendimiento bajo carga y la observabilidad no son atributos independientes del software, sino dimensiones profundamente interconectadas que deben validarse de forma empírica y no darse nunca por sentadas.

Referencias

- [1] ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices. Software engineering code of ethics and professional practice. *IEEE Computer*, 32(10):84–89, 1999. DOI: <https://doi.org/10.1109/MC.1999.796142>.
- [2] B. Burns. *Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services*. O'Reilly Media, 2 edition, 2025.
- [3] R. T. Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, Irvine, CA, USA, 2000.
- [4] R. Jain. *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. Wiley, 1991.
- [5] M. B. Jones, J. Bradley, and N. Sakimura. JSON Web Token (JWT). Technical Report RFC 7519, IETF, 2015.
- [6] M. Kleppmann. *Designing Data-Intensive Applications*. O'Reilly Media, Sebastopol, CA, USA, 2017.
- [7] S. Newman. *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media, Sebastopol, CA, USA, 2 edition, 2021.